



Pflichtenheft

Desktop Secret Manager / lokale Tresor-Anwendung im SASD-Stil

Umsetzungs- und Ausführungsdokument für die Realisierung einer lokalen Windows-Desktop-Anwendung zur Verwaltung verschlüsselter Tresore, strukturierter Secrets und einer kontrollierten Password-Safe-Interop.

Metadaten	Dokumentstand
Dokumenttyp	Pflichtenheft
Projektgegenstand	Lokale Windows-Desktop-Anwendung zur Verwaltung verschlüsselter Tresore, strukturierter Secrets und zusätzlicher Password-Safe-Kompatibilität
Bezugsbasis	Vollständiger Projektdialog sowie das daraus abgeleitete Lastenheft und Architekturkonzept
Zielformat	Windows Desktop, Single-User, offline-first
Technologierahmen	C# / .NET 8 / WinForms; interne Schichten für Domain, Application, Security, Storage, Interop und UI
Zieldokument	Verbindliche Ausführungsspezifikation für V1, V1.x und V2.0
Version	1.0
Datum	22.04.2026

Leserhinweis

Dieses Pflichtenheft beschreibt nicht mehr das fachliche Zielbild allein, sondern die konkrete Ausführung der Anwendung. Es legt fest, wie Datenmodell, Sicherheit, Dateiverwaltung, Mehrtresor-Konzept, Bedienoberfläche, Password-Safe-Interop, Teststrategie und Release-Stufen umgesetzt werden. Bewusst vertagte Themen bleiben dokumentiert, werden aber nicht stillschweigend in V1 hineingezogen.

Management Summary

Dieses Pflichtenheft konkretisiert den zuvor erarbeiteten Projektstand in eine belastbare Umsetzungsspezifikation. Realisiert wird eine lokale Windows-Desktop-Anwendung in C#, die mehrere voneinander unabhängige verschlüsselte Tresore verwaltet, strukturierte Secrets technisch sauber abbildet und zusätzlich einen kontrollierten Import aus Password Safe unterstützt.

Der Kern der Lösung ist bewusst nicht auf klassische Web-Logins beschränkt. Einträge dürfen auch Datenbankverbindungen, Mail-Konten, FTP- oder SFTP-Zugänge, Hosting-Zugänge, Server- oder API-Secrets sowie frei definierte technische Zusatzfelder umfassen. Gerade für den SASD-Kontext ist entscheidend, dass Zugangsdaten nicht isoliert, sondern mit ihrem fachlichen und technischen Nutzungskontext verwaltet werden.

Die Anwendung wird als Single-User-Produkt mit starkem Sicherheitsfokus ausgeführt. V1 bleibt lokal, ohne Cloud-Synchronisation, ohne Browserintegration, ohne Team-Sharing und ohne Anhänge. Im Gegenzug werden Datenmodell, Dateiformat, Schreibstrategie, Sperrlogik, Auto-Lock, Zwischenablage-Schutz, Testbarkeit und Password-Safe-Interop von Anfang an sauber und konservativ angelegt.

1. Dokumentzweck, Bezugsbasis und Arbeitsweise

Dieses Dokument beschreibt die vom Auftragnehmer verbindlich zu liefernde Lösung. Es übersetzt das Lastenheft in eine konkrete Ausführung mit Implementierungsentscheidungen, Release-Grenzen, Datenstrukturen, Bedienfluss und Qualitätsmaßnahmen. Die Bezugsbasis ist der vollständige Projektdialog einschließlich der darin getroffenen Grundsatzentscheidungen zu Datenmodell, Sicherheit, Mehrtresor-Verwaltung und Interop.

Das Dokument folgt dem SASD-Dokumentstil: ruhiges Layout, klare Kapitelhierarchie, gut lesbare Prosa, sparsame aber belastbare Tabellen sowie explizite Nennung von bewussten Nicht-Zielen. Entscheidungen werden so dokumentiert, dass die spätere Umsetzung, Testbarkeit und Wartbarkeit nicht von losem Chat-Kontext abhängen.

Arbeitsgrundsatz

V1 wird klein, robust und sicherheitsbewusst gehalten. Neue Komfort- oder Prestige-Funktionen werden nur dann in den Umfang aufgenommen, wenn sie das Fundament nicht gefährden. Interoperabilität mit Password Safe wird additiv umgesetzt und darf das interne Datenmodell nicht dominieren.

2. Zielsystem und bewusste Abgrenzung

Das Zielsystem ist eine lokale Desktop-Anwendung für Windows 10 und 11. Die Anwendung arbeitet offline-first, speichert ihre fachlichen Daten in verschlüsselten Tresor-Dateien und kann mehrere voneinander unabhängige Tresore wie etwa Privat, SASD intern, IONOS, Kundenprojekt oder Archiv getrennt verwalten.

Nicht Bestandteil des V1-Grundumfangs sind Browser-Autofill, Cloud-Sync, Team-Vaults, mobile Clients, Plugins, Web-Frontend, TOTP-Verwaltung, echte Binäranhänge, Multi-User-Rechteverwaltung oder eine vollständige roundtrip-stabile Bearbeitung nativer .psafe3-Dateien. Diese Themen bleiben dokumentiert, werden aber bewusst nicht in den initialen Lieferumfang aufgenommen.

3. Release- und Lieferstrategie

Die Umsetzung erfolgt gestuft. V1 liefert das sichere Fundament. V1.x härtet das Produkt, erweitert die Kompatibilität und verbessert Bedienung und Migrationsfähigkeit. V2.0 ist für weitergehende Interop- und Produktivitätsfunktionen vorgesehen.

Release	Verbindlicher Umfang
V1	Lokale Tresore, internes Format .svault, Gruppen, Tags, Eintragstypen, strukturierte Zusatzfelder, Suche, Passwortgenerator, Auto-Lock, Clipboard-Clear, atomisches Speichern, verschlüsselte Backups, kontrollierter .psafe3-Import
V1.x	Härtung, .psafe3-Export für sauber abbildbare Inhalte, Kompatibilitätsberichte, Vorlagen für technische Eintragstypen, verbesserte Migrations- und Konfliktbehandlung, read-only Zweitöffnung
V2.0	Erweiterter Kompatibilitätsmodus, Multi-Vault-Suche, Compare/Merge, TOTP, optionale Anhänge, weitergehende Komfort- und Analysefunktionen

4. Technologischer Lösungsrahmen

Die Anwendung wird als C#-Desktop-Anwendung auf Basis von .NET 8 und WinForms umgesetzt. Diese Wahl ist für das Vorhaben zweckmäßig, weil sie kurze Wege zu einer robusten Windows-Oberfläche erlaubt, ohne eine übermäßig komplexe UI-Infrastruktur einzuführen. Zugleich wird die Lösung architektonisch so gebaut, dass Fachdaten, Sicherheit, Persistenz, Interop und UI sauber getrennt bleiben.

Der Quellcode ist stark kommentiert zu liefern. Neben XML-Dokumentationskommentaren mit `///` für Klassen, Methoden und zentrale Modelle sind im Implementierungscode erklärende `//-`Kommentare zu setzen, sofern sie für spätere Wartung und Verständnis nützlich sind. Kommentarstil und Benennung müssen konsistent und professionell bleiben.

Die Ordner- und Namensstruktur folgt den SASD Engineering Standards. Sicherheits- und Architekturentscheidungen werden vor dem Codieren bewusst getroffen; V1 bleibt klein und robust; Domänenlogik wird nicht in die Forms verschoben; Dokumentation und Code halten dieselben Begriffe und dieselbe Schichtenlogik ein.

5. Zielarchitektur im Überblick

Die Zielarchitektur trennt fachliches Modell, Anwendungslogik, Sicherheitsfunktionen, Dateipersistenz, Password-Safe-Interop, Oberfläche und Tests. Dadurch bleibt die Lösung erweiterbar, obwohl V1 funktional bewusst begrenzt ist.

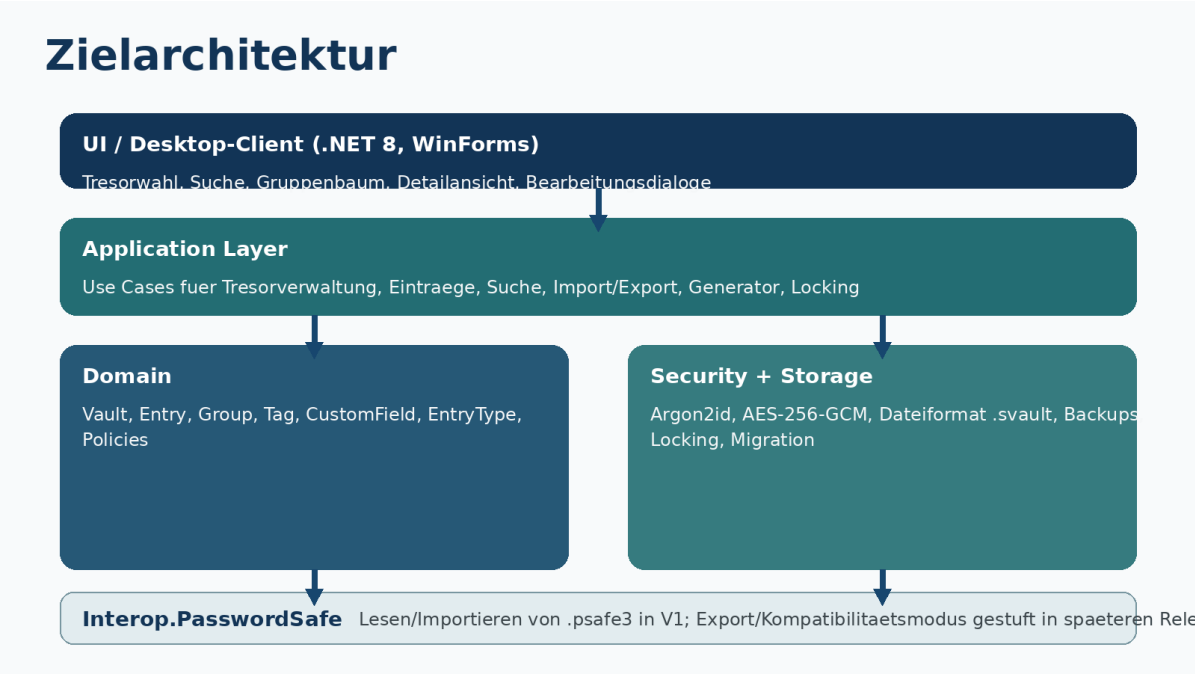


Abbildung 1: Zielarchitektur mit den geplanten Hauptschichten der Anwendung.

Modul	Verantwortung
Domain	Vault, Entry, Group, Tag, CustomField, Enumerationen und fachliche Regeln ohne UI-Abhängigkeit
Application	Use Cases und Services für Tresor-Lebenszyklus, Eintragsverwaltung, Suche, Import/Export, Passwortgenerator
Security	Key Derivation, Verschlüsselung, Lock-Verhalten, Redaction/Maskierung, Clipboard-Schutz, sicherheitsnahe Hilfsfunktionen
Storage	Lesen, Schreiben, Dateiversionierung, atomische Speicherung, Backup-Logik, Lockfile/Dateisperre, Migrationen
Interop.PasswordSafe	Reader, Mapper, spätere Writer-Komponente, Kompatibilitätsbericht, Import-Assistent
UI	WinForms-Hauptfenster, Editierdialoge, Such- und Filterbedienung, Fehlermeldungen und sichere Darstellung
Tests	Unit-, Integrations- und Negativtests für Krypto, Persistenz, Mapping, Suche und Dateifehler

6. Fachliches Datenmodell

Das interne Datenmodell ist führend. Es wird nicht an den Grenzen des Fremdformats .psafe3 ausgerichtet, sondern beschreibt fachlich sauber, was die Anwendung selbst verwalten muss. Interop arbeitet über Mapping auf dieses Modell und nicht umgekehrt.

Fachliche Haupteinheiten sind Vault, Entry, Group, Tag und CustomField. Ein Vault enthält die Gruppenstruktur, die Einträge, die Metadaten des Tresors sowie nicht-geheime Einstellungen, die fachlich zum Tresor gehören. Ein Entry beschreibt genau einen primären Zugang oder ein primäres Secret. Gruppen bilden die feste Ablagestruktur; Tags schaffen flexible Querbezüge.

Objekt	Verbindliche Umsetzung
Vault	Eindeutige Tresor-ID, Anzeigename, Formatversion, Eintragsliste, Gruppenbaum, Tag-Vorrat, tresorbezogene Metadaten
Entry	Eindeutige ID, Eintragstyp, Titel, Benutzername oder Principal, Secret, Notizen, Hauptgruppe, Tags, Zusatzfelder, Zeitstempel, Status
Group	Hierarchischer Knoten mit Name, Parent-Bezug und optionalen Darstellungsmetadaten
Tag	Freies, normalisiertes Schlagwort zur Mehrfachzuordnung über Gruppen hinaus
CustomField	Feldname, Feldwert, FieldKind, Kennzeichnung als geheim oder nicht geheim, Sortierreihenfolge

6.1 Eintragstypen

Mindestens die folgenden Eintragstypen sind vorzusehen: Login, Database, Mail, Server, FTP/SFTP, API, License, SecureNote und Custom. Die Typen dienen sowohl der fachlichen Einordnung als auch der UI-Darstellung. Sie schränken Zusatzfelder nicht starr ein, sondern steuern Pflichtgefühl, Vorbelegung und Sichtlogik.

Typ	Typische fachliche Nutzung
Login	Klassische Online- oder Portalzugänge mit URL, Benutzername, Passwort, Notizen
Database	Host, Port, Datenbankname, Schema, Benutzername, Passwort, SSL/TLS-Hinweise
Mail	E-Mail-Adresse, IMAP/SMTP-Hosts, Ports, TLS, Benutzername, Passwort
Server	Hostname oder IP, Protokoll, Port, Benutzer, Passwort oder Schlüsselhinweis
FTP/SFTP	Host, Port, Protokoll, Zielpfad, Benutzer, Passwort
API	Client-ID, Secret, Token, Endpoint, Scope- oder Rechtehinweise
License	Produkt, Lizenzschlüssel, Hersteller, Ablauf- oder Vertragsbezug
SecureNote / Custom	Freie oder ungewöhnliche Inhalte, die nicht sinnvoll in ein starres Schema passen

6.2 Zusatzfelder

Zusatzfelder sind Kernfunktion und kein späterer Hack. Sie ermöglichen technische und organisatorische Informationen wie Host, Port, Datenbankname, Mandant, Umgebung, Projekt, Provider, Zertifikatshinweis, SMTP-Server, Tenant oder Deployment-Zweck. Jedes Zusatzfeld erhält einen Feldtyp. Vorgesehen sind mindestens Text, Secret, Number, URL, Hostname, Port, E-Mail, Date, Boolean und freier Typ.

Geheime Zusatzfelder sind standardmäßig maskiert, werden beim Kopieren wie andere Secrets behandelt und dürfen weder im Suchindex unverschlüsselt noch im Log im Klartext auftauchen. Nicht geheime Zusatzfelder dürfen durchsuchbar und in Listenansichten selektiv anzeigbar sein.

7. Bedien- und UI-Konzept

Die Hauptoberfläche gliedert sich in drei funktionale Zonen. Links befindet sich der Gruppenbaum. Oben stehen Suche, Typfilter, Tagfilter und gegebenenfalls Schnellfilter. Im Hauptbereich werden die Einträge der aktuellen Auswahl gelistet. Ein Detailbereich zeigt die vollständigen Informationen des markierten Eintrags an, ohne dass geheime Inhalte ungefragt sichtbar werden.

Einträge werden in separaten Dialogen oder klar abgegrenzten Bearbeitungsansichten editiert. Die UI muss den Eintragstyp berücksichtigen und Zusatzfelder sauber verwalten können. Secret-Felder erhalten Ansehen-/Verbergen- und Kopieren-Schaltflächen. Doppelklick, Kontextmenüs und Tastaturbedienung sind zu unterstützen, sofern sie die Sicherheitslogik nicht unterlaufen.

UI-Bereich	Pflichtverhalten
Gruppenbaum	Hierarchische Navigation und organisatorische Ablage
Tag- und Typfilter	Schnelles Auffinden über Querbezüge
Eintragsliste	Übersicht der Treffer mit sicherer Standarddarstellung ohne Secret-Klartext
Detailansicht	Vollständige fachliche Sicht mit explizitem Offenlegen oder Kopieren geheimer Felder
Editierdialog	Anlegen, Bearbeiten, Archivieren und Löschen von Einträgen einschließlich Zusatzfelder

UI-Grundsatz

Ordnung entsteht über Gruppen, Flexibilität über Tags, Vollständigkeit über Zusatzfelder. Die Oberfläche darf diese drei Konzepte nicht vermischen, sondern muss sie sichtbar als unterschiedliche Werkzeuge behandeln.

8. Sicherheitskonzept und Schutzgrenzen

Das Produkt ist als sicherheitskritische Einzelplatz-Anwendung ausgelegt. Die Anwendung soll wirksam gegen Diebstahl oder Verlust der Tresor-Datei, gegen unbefugtes Lesen der Daten und gegen versehentliche Offenlegung durch schlechte Bedienung schützen. Nicht zugesagt wird ein Schutz gegen bereits kompromittierte Betriebssysteme, Malware, Keylogger, aggressive RAM-Auslese oder andere Angriffe, bei denen das Ausführungssystem bereits nicht vertrauenswürdig ist.

Die Zielkonfiguration für produktive Releases lautet Argon2id als KDF und AES-256-GCM als authentifizierte Verschlüsselung. Pro Tresor werden individuelle Salt- und Nonce-Werte verwendet. KDF-Parameter sind im Dateiformat mitzuführen, damit spätere Erhöhungen ohne stillen Formatbruch möglich bleiben. Ein produktiver Release nur mit PBKDF2 ist nicht vorgesehen; ein temporärer Entwicklungs-Fallback wäre allenfalls intern und dokumentationspflichtig.

Bereich	Verbindliche Ausführung
Master-Passwort	Pro Tresor eigenständig; keine Speicherung außerhalb des entschlüsselten Arbeitskontexts
Secrets im UI	Standardmäßig verdeckt; Offenlegung nur explizit und temporär

Bereich	Verbindliche Ausführung
Zwischenablage	Kopieren wird unterstützt; Auto-Clear nach definierbarer Zeit ist Pflicht
Auto-Lock	Sperre nach Inaktivität; erneute Entsperrung nur mit Master-Passwort des aktiven Tresors
Logging	Keine Passwörter, Tokens oder geheimen Zusatzfelder im Klartext loggen
Recovery	Keine Hintertür, kein versteckter Recovery-Modus, kein stiller Notzugang

8.1 Sicherheit im laufenden Betrieb

Secrets werden nur so lange im Klartext gehalten, wie die Bedienung es erfordert. Beim Sperren des Tresors sind sichtbare Detailfelder zu leeren oder zurück in die maskierte Form zu versetzen. Fehlermeldungen dürfen keine sensiblen Inhalte einschließen. Temporäre Klartextdateien sind unzulässig. Wo eine vollständige Sicherheit im RAM mit .NET nicht realistisch ist, wird dies offen als Best-Effort-Grenze dokumentiert und nicht verschleiert.

9. Vault-Dateiformat und Dateiverwaltung

Das Primärformat der Anwendung ist ein eigenes internes Tresorformat mit der Dateiendung .svault. Das Format trägt eine klare Magic-Header-Kennung, eine Formatversion, KDF-Parameter, Salt, Nonce, Auth-Tag sowie den verschlüsselten Payload. Der Payload enthält das interne Datenmodell des Tresors in einer versionierten Struktur.

Speichervorgänge erfolgen atomisch. Die Anwendung schreibt zunächst in eine temporäre Datei, synchronisiert den Inhalt und ersetzt dann die bestehende Tresor-Datei kontrolliert. Optional werden rotierende, ebenfalls verschlüsselte Backup-Dateien erzeugt. Beschädigte Schreibstände dürfen nicht als gültiger Tresor erscheinen.

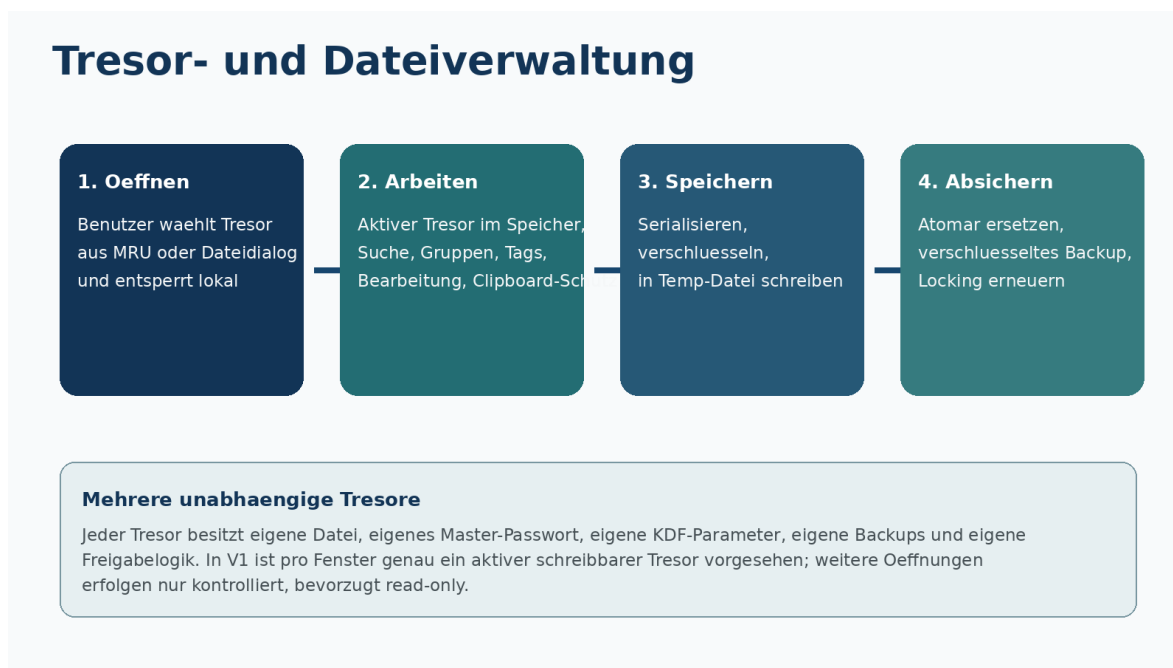


Abbildung 2: Konzept des internen Tresorformats mit Verschlüsselung, Versionierung und Schutz vor Halbspeicherung.

Aspekt	Pflichtverhalten
Dateiendung	.svault als Primärformat der Anwendung
Dateiinhalt	Header mit Versions- und Sicherheitsparametern plus verschlüsselter Nutzlast
Schreiben	Atomisch über temporäre Datei und kontrollierten Austausch
Backups	Optional, rotierend und weiterhin verschlüsselt
Migrationen	Explizite Format- und Datenmigration mit Versionsprüfung
Beschädigung	Saubere Fehlerbehandlung, keine stillschweigende Teilrettung als Erfolgsmeldung

10. Mehrtresor-Konzept

Mehrere voneinander unabhängige Tresore sind kein Randthema, sondern zentraler Teil des Produkts. Die Anwendung muss getrennte Tresore für private, interne, providerbezogene, kundenbezogene oder archivierte Zugänge unterstützen. Die Trennung erfolgt nicht nur optisch, sondern datei- und passwortseitig.

In V1 darf pro Programmfenster genau ein aktiver, schreibbarer Tresor geöffnet sein. Mehrere Tresor-Dateien dürfen im Gesamtsystem existieren und nacheinander geöffnet werden. Eine parallele Zweitöffnung desselben Tresors wird konservativ behandelt, vorzugsweise über Lockfile oder Dateisperre. Bei Konflikten hat Sicherheit Vorrang vor Komfort.

Mehrtresor-Aspekt	Ausführung
Beispiele	Privat, SASD intern, IONOS, Kunde A, Archiv 2025
MRU-Liste	Lokale Liste zuletzt verwendeter Tresore ohne Speicherung von Master-Passwörtern
Zweitöffnung	Konflikterkennung; in V1 keine parallele schreibende Bearbeitung derselben Datei
Konfiguration	Nur nicht geheime App-Einstellungen außerhalb des Tresors

11. Password-Safe-Interop

Die Interoperabilität mit Password Safe wird als Zusatzschicht umgesetzt. Das interne Datenmodell bleibt führend. Password Safe ist ein wichtiges Kompatibilitätsziel, aber nicht der Maßstab, der das interne Modell fachlich verengt.

V1 liefert einen kontrollierten Import aus .psafe3. Dieser Import liest vorhandene Password-Safe-Dateien ein, validiert die Datei, mappt abbildbare Inhalte in das interne Modell und weist nicht sauber abbildbare Informationen sichtbar aus. Es findet kein stillschweigender Datenverlust statt.

Ein Export nach .psafe3 wird bewusst erst in V1.x eingeführt, weil er eine belastbare Mapping-Strategie, Verlusthinweise und Kompatibilitätsberichte voraussetzt. Ein direkter Bearbeitungsmodus für .psafe3 als primäres Arbeitsformat ist frühestens für V2.0 vorgesehen.

Baustein	Geplante Ausführung
Import V1	Reader + Mapper + Import-Assistent + Importbericht
Export V1.x	Writer für den klar abbildbaren Teil plus Kompatibilitätsbericht
Kompatibilitätsmodus V2.0	Weitergehender Umgang mit .psafe3, ohne das interne Format aufzugeben
Grundsatz	Unbekannte oder nicht exakt abbildbare Inhalte werden markiert, nicht verschluckt

11.1 Feld-Mapping-Grundsätze

Direkt abbildbare Standardinformationen wie Titel, Benutzername, Passwort, URL, Gruppe und Notizen werden strukturiert übernommen. Bekannte Zusatzinformationen werden in CustomFields transformiert. Informationen, die semantisch nicht sauber in das interne Modell passen, werden im Importbericht dokumentiert und falls sinnvoll als gekennzeichneteter Metadatenblock an den Eintrag gehängt. Damit bleibt nachvollziehbar, welche Inhalte originär aus Password Safe stammen und welche im Zielsystem erweitert wurden.

12. Funktionsspezifikation V1

Die nachfolgenden Pflichten sind für V1 verbindlich zu realisieren. Sie beschreiben nicht nur fachliche Anforderungen, sondern die konkrete Art ihrer Ausführung.

ID	Verbindliche V1-Pflicht
PF-01	Neue .svault-Tresore anlegen, bestehende Tresore öffnen, speichern, schließen und wieder entsperren
PF-02	Eintragsverwaltung mit Typen, Hauptgruppe, Tags, Notizen, Zeitstempeln und strukturierten Zusatzfeldern
PF-03	Sichere Standarddarstellung geheimer Werte mit explizitem Anzeigen und Kopieren
PF-04	Suche über Titel, Gruppen, Tags, Typen, Notizen und nicht geheime Zusatzfelder
PF-05	Passwortgenerator mit Längen- und Zeichensatzoptionen
PF-06	Auto-Lock nach Inaktivität und Clipboard-Auto-Clear
PF-07	Atomisches Speichern, verschlüsselte Backups, Konflikterkennung bei Zweitöffnung
PF-08	MRU-Liste ohne geheime Inhalte und ohne Speicherung von Master-Passwörtern
PF-09	Kontrollierter .psafe3-Import mit Bericht
PF-10	Unit- und Integrationstests für Krypto, Storage, Suche und Import

13. Nichtfunktionale Spezifikation

Die Anwendung muss ruhig, nachvollziehbar und im Alltag belastbar wirken. Startzeit, Filterreaktion und normale Bedienoperationen sollen ohne spürbare Trägheit erfolgen, solange die Tresorgröße in einem typischen Einzelplatzrahmen bleibt. Fehlermeldungen sollen verständlich, aber nicht plaudernd sein.

Qualitätsziel	Verbindliche Ausführung
Wartbarkeit	Klare Schichtentrennung, stabile Namensgebung, starke Kommentierung, keine Geschäftslogik in Form-Klassen
Sicherheit	Keine Secret-Leaks in Log, Export, Fehlermeldungen, Cache oder Klartext-Temp-Dateien
Benutzbarkeit	Sichere Standardwerte, sichtbare Gruppierung und Tags, konsistente Dialoge, tastaturfreundlich
Erweiterbarkeit	Vorbereitung auf V1.x/V2.0 ohne Vorwegimplementierung unnötiger Funktionen
Dokumentation	Quellcode und fachliche Dokumentation müssen dieselben Begriffe und Release-Grenzen verwenden

14. Entwicklungs-, Test- und Dokumentationskonzept

Die Entwicklung folgt einem sicherheitsbewussten, inkrementellen Vorgehen. Erst werden Datenmodell, Sicherheitsfunktionen, Persistenz und grundlegende UI-Flows stabilisiert, danach Komfort- und Interop-Erweiterungen.

Zu liefern sind Unit-Tests für Key Derivation, Verschlüsselung, Dechiffrierung, Serialisierung, Suchlogik und Mapping. Hinzu kommen Integrationstests für Anlegen, Speichern, Wiederöffnen, Backup-Erzeugung, beschädigte Dateien, falsche Passwörter und .psafe3-Importfälle. Negativtests sind ausdrücklich Teil des Umfangs.

Neben dem Code sind technische Begleitdokumente zu pflegen: Datenmodellbeschreibung, Dateiformatbeschreibung für .svault, Interop-Mapping-Regeln, Testübersicht und Änderungsprotokoll zu V1.x/V2.0-Erweiterungen. Die Dokumentation soll nicht nur vorhanden, sondern für spätere Wartung wirklich nutzbar sein.

Testebene	Pflichtumfang
Unit-Tests	KDF, AES-GCM, Model-Serialisierung, Such- und Filterlogik, Mapper
Integrationstests	Tresor anlegen, speichern, schließen, neu öffnen, Backup, Lock-Verhalten
Negativtests	Falsches Passwort, beschädigte Datei, abgeschnittene Datei, unbekannte Felder, Importfehler
Manuelle QA	UI-Flows, sichere Standarddarstellung, Copy/Lock-Verhalten, Fehlerdialoge

15. Abnahmekriterien für V1

V1 gilt als abnahmefähig, wenn ein neuer Tresor sicher angelegt, mit Einträgen aus mehreren Typen gefüllt, geschlossen, erneut geöffnet und ohne Datenverlust weiterbearbeitet werden kann. Mehrere unabhängige Tresore müssen getrennt nutzbar sein. Secrets dürfen im normalen UI-Betrieb nicht unbeabsichtigt sichtbar werden.

ID	Abnahmekriterium
AK-01	Neuer .svault-Tresor kann erstellt, mit Master-Passwort geschützt und gespeichert werden

ID	Abnahmekriterium
AK-02	Einträge mit Gruppen, Tags und Zusatzfeldern lassen sich anlegen, bearbeiten, archivieren und löschen
AK-03	Secrets bleiben standardmäßig verborgen; Kopieren und Anzeigen funktionieren kontrolliert
AK-04	Auto-Lock und Clipboard-Auto-Clear funktionieren reproduzierbar
AK-05	Speichern erfolgt atomisch; Backup-Datei wird korrekt erzeugt, sofern aktiviert
AK-06	.psafe3-Import führt Daten kontrolliert über und erzeugt nachvollziehbaren Bericht
AK-07	Testsuite deckt Kernbereiche ab und läuft ohne offene kritische Fehler

16. Bewusst vertagte Themen

Die folgenden Themen bleiben außerhalb von V1, obwohl sie fachlich interessant oder später sinnvoll sind: Browserintegration, Team-Sharing, Cloud-Synchronisation, Mobile Apps, TOTP, Binäranhänge, Mehrbenutzerrechte, globale Multi-Vault-Suche, Compare/Merge, direkte .psafe3-Bearbeitung als primäres Arbeitsformat und erweiterte Analyse- oder Reporting-Funktionen.

Diese Vertagung ist kein Weglassen aus Versehen, sondern eine bewusste Maßnahme zur Stabilisierung der ersten liefersicheren Version. Nur so bleibt das Fundament für spätere Erweiterungen tragfähig.

17. Risiken und Maßnahmen

Risiko	Gegenmaßnahme
Zu starres Datenmodell	Führendes internes Modell mit Eintragstypen und CustomFields; keine Ausrichtung am Fremdformat allein
Unsichere Dateiverwaltung	Atomisches Speichern, Lockfile/Dateisperre, verschlüsselte Backups, Versionsprüfung
Zu frühe Interop-Komplexität	V1 nur Import; Export und Kompatibilitätsmodus bewusst später
Secret-Leaks im Alltag	Maskierung, Clipboard-Clear, Auto-Lock, Redaction-Regeln für Logging und Fehlertexte
UI-Überladung	V1 hält Oberfläche auf Gruppen, Tags, Suche, Details und Editierdialoge begrenzt

18. Schlussbemerkung

Mit diesem Pflichtenheft liegt eine umsetzbare, in sich konsistente und bewusst konservative Spezifikation vor. Sie verbindet die fachliche Breite des Vorhabens mit einem realistischen Sicherheits- und Release-Rahmen. Das Dokument ist so angelegt, dass daraus in einem nächsten Schritt direkt Architektur-Feinschnitt, Arbeitspakete, C#-Projektstruktur und Implementierung abgeleitet werden können.

Anhang A – Release-Matrix

Funktion	V1	V1.x	V2.0
Interne .svault-Tresore	Ja	Ja	Ja
Gruppen und Tags	Ja	Ja	Ja
Strukturierte Zusatzfelder	Ja	Ja	Ja
Kontrollierter .psafe3-Import	Ja	Ja	Ja
.psafe3-Export	Nein	Ja	Ja
Direkter Kompatibilitätsmodus	Nein	Nein	Ja
Mehrere aktive Tresore parallel	Ein aktiver Schreib-Tresor pro Fenster	Read-only Zweitöffnung möglich	Erweitert
TOTP	Nein	Nein	Ja
Anhänge	Nein	Nein	Optional

Anhang B – Verbindliche Struktur des C#-Projekts

Projektordner	Aufgabe
Domain	Modelle und fachliche Regeln
Application	Use Cases und Services
Security	Krypto, Lock, Clipboard-Schutz
Storage	Dateiformat, Lesen/Schreiben, Backups, Migrationen
Interop.PasswordSafe	Import/Export und Kompatibilitätsberichte
UI	WinForms-Oberfläche
Tests	Automatisierte Qualitätssicherung

Anhang C – Vorgaben für Quellcode und Kommentare

Der Quellcode ist so zu schreiben, dass spätere Einarbeitung, Wartung und Erweiterung durch den Auftraggeber realistisch bleiben. Öffentliche und zentrale interne Typen erhalten XML-Kommentare mit ///. Algorithmisch oder architektonisch relevante Stellen erhalten zusätzliche //-Kommentare, soweit sie dem Verständnis dienen. Kommentartexte erklären Absicht und Grund, nicht nur den offenkundigen Befehl.